NUnit Hands-On – Answers

1. CalcLibrary – Source Code (Class Under Test)

The CalcLibrary project is downloaded and opened in Visual Studio. It contains MathLibrary.cs which is the class we will write all tests against. The key points are:

- Add, Subtract, Multiply, Divide operations that return a double.
- A private field 'result' that stores the last computed value.
- A public property GetResult that exposes 'result'.
- A void method AllClear() that resets 'result' to 0.
- Divide throws ArgumentException when divisor is 0.

```
using System;

namespace CalcLibrary {
public class MathLibrary
{ private double result;

public double GetResult
{ get { return result; }
}

public double Add(double a, double b)
{ result = a + b; return result; }

public double Subtract(double a, double b)
{ result = a - b; return result; }

public double Multiply(double a, double b)
{ result = a * b; return result; }

public double Divide(double a, double b) { if (b
== 0) throw new ArgumentException("Division by
zero"); result = a / b; return result;
}
```

```
// Void method - resets stored result to 0
public void AllClear() { result = 0; }
}
}
```

2. Test Project Setup

Create the test project

- In Visual Studio: File → Add → New Project → Class Library (C#).
- Name it CalcLibrary.Tests.
- Delete the default Class1.cs.
- Add a project reference to CalcLibrary.

Install NuGet packages

Package Manager Console

```
Install-Package NUnit
Install-Package NUnit3TestAdapter
Install-Package Microsoft.NET.Test.Sdk
```

Add test class – MathLibraryTests.cs

CalcLibrary.Tests/MathLibraryTests.cs – skeleton

```
using System; using
NUnit.Framework;
using CalcLibrary;

namespace CalcLibrary.Tests
{ [TestFixture] public class
MathLibraryTests { private
MathLibrary _math;

// Runs before every test - gives a clean MathLibrary each
time [SetUp] public void Init() { _math = new MathLibrary();
}

// --- test methods will go here ---
}
}
```

3. Parameterized Test – Subtraction

Each [TestCase(...)] line passes a different set of arguments to the same test method. NUnit runs the method once per TestCase and shows each as a separate entry in Test Explorer.

MathLibraryTests.cs – Subtract

```
[TestCase(10, 3, 7)]
[TestCase(20, 10, 10)]
[TestCase(0, 5, -5)] [TestCase(-4, -2, -2)] public void
TestSubtract(double a, double b, double expected) { double
actual = _math.Subtract(a, b); Assert.AreEqual(expected,
actual);
}
```

[Screenshot – Test Explorer: Subtraction (all 4 passed)]

```
Test Explorer - Subtraction

Passed TestSubtract(10, 3, 7) 8 ms
Passed TestSubtract(20, 10, 10) 5 ms
Passed TestSubtract(0, 5, -5) 4 ms
Passed TestSubtract(-4, -2, -2) 4 ms

4 / 4 passed
```

4. Parameterized Test – Multiplication

Same pattern as Subtract – multiple TestCase attributes, one test method.

MathLibraryTests.cs – Multiply

```
[TestCase(3, 4, 12)]
[TestCase(5, 5, 25)]
[TestCase(-2, 3, -6)] [TestCase(0, 10, 0)] public void
TestMultiply(double a, double b, double expected) { double
actual = _math.Multiply(a, b); Assert.AreEqual(expected,
actual);
}
```

[Screenshot – Test Explorer: Multiplication (all 4 passed)]

```
Test Explorer – Multiplication

Passed TestMultiply(3, 4, 12) 4 ms
Passed TestMultiply(5, 5, 25) 3 ms
Passed TestMultiply(-2, 3, -6) 3 ms
Passed TestMultiply(0, 10, 0) 3 ms

4 / 4 passed
```

5. Parameterized Test – Division (including divide by zero)

Normal division cases use TestCase exactly like Subtract and Multiply. The divide-by-zero case needs special handling:

- We pass divisor = 0 to Divide().
- We wrap the call in a try block.
- If NO exception is thrown, we call Assert.Fail("Division by zero") so the test fails and that message appears in TestExplorer.
- If an ArgumentException IS thrown, the catch block runs and we verify the exception message with Assert.AreEqual.

MathLibraryTests.cs – Divide (normal cases)

```
[TestCase(10, 2, 5)]
[TestCase(9, 3, 3)] [TestCase(-6, 2, -3)] public void
TestDivide(double a, double b, double expected) { double
actual = _math.Divide(a, b); Assert.AreEqual(expected,
actual);
}
```

MathLibraryTests.cs – Divide by zero (try/catch + Assert.Fail)

```
[TestCase] public void TestDivide_ByZero_ThrowsException()
{ try { // Divisor is 0 - MathLibrary should throw
ArgumentException
_math.Divide(10, 0);
// If we reach this line, no exception was thrown.
// That means the guard is missing - fail the
test. Assert.Fail("Division by zero"); } catch
(ArgumentException ex) {
// Exception was thrown as expected - verify the message
Assert.AreEqual("Division by zero", ex.Message);
}
}
```

[Screenshot – Test Explorer: Division (all 4 passed)]

```
Test Explorer - Division

Passed TestDivide(10, 2, 5) 5 ms
Passed TestDivide(9, 3, 3) 4 ms
Passed TestDivide(-6, 2, -3) 3 ms
Passed TestDivide_ByZero_ThrowsException
6 ms ArgumentException caught, message =
"Division by zero" OK

4 / 4 passed
```

6. Testing a Void Method – TestAddAndClear

AllClear() is void so it returns nothing. We test it indirectly through the GetResult property.

- Call Add(5, 3) and assert that GetResult is 8.
- Call AllClear().
- Assert that GetResult is now 0.

MathLibraryTests.cs – TestAddAndClear

```
[TestCase] public void TestAddAndClear() {
// Step 1: perform addition and check
result
_math.Add(5, 3); Assert.AreEqual(8, _math.GetResult,
"Add(5,3) should give 8");
// Step 2: call the void method under test
_math.AllClear();
// Step 3: verify result was reset to 0 Assert.AreEqual(0,
_math.GetResult, "After AllClear() result should be 0");
}
```

```
Test Explorer - Void Method
Passed TestAddAndClear 7 ms
Assert 1: GetResult == 8 OK Assert 2:
GetResult == 0 OK (after AllClear())
1 / 1 passed
```

7. Complete Test File

```
using System; using
NUnit.Framework;
using CalcLibrary;

namespace CalcLibrary.Tests
{ [TestFixture] public class
MathLibraryTests { private
MathLibrary _math;

[SetUp] public void
Init() { _math = new
MathLibrary();
}

// ■■ Subtraction
[TestCase(10, 3, 7)]
[TestCase(20, 10, 10)]
[TestCase(0, 5, -5)] [TestCase(-4, -2, -2)] public void
TestSubtract(double a, double b, double expected) { double
actual = _math.Subtract(a, b); Assert.AreEqual(expected,
actual);
}

// ■■ Multiplication
[TestCase(3, 4, 12)]
[TestCase(5, 5, 25)]
[TestCase(-2, 3, -6)] [TestCase(0, 10, 0)] public void
TestMultiply(double a, double b, double expected) { double
actual = _math.Multiply(a, b); Assert.AreEqual(expected,
actual);
}

// ■■ Division - normal cases
[TestCase(10, 2, 5)]
[TestCase(9, 3, 3)] [TestCase(-6, 2, -3)] public void
TestDivide(double a, double b, double expected) { double
actual = _math.Divide(a, b);
```

```
Assert.AreEqual(expected, actual);
}

// ■■ Division - by zero
[TestCase] public void
TestDivide_ByZero_ThrowsException() { try {
_math.Divide(10, 0);
// Should not reach here Assert.Fail("Division
by zero"); } catch (ArgumentException ex) {
Assert.AreEqual("Division by zero",
ex.Message);
}
}

// ■■ Void method AllClear
[TestCase] public void
TestAddAndClear() {
_math.Add(5, 3); Assert.AreEqual(8, _math.GetResult,
"Add(5,3) should give 8");
_math.AllClear();
Assert.AreEqual(0, _math.GetResult, "After AllClear() result should be 0");
}
}
}
```

8. Final Test Run – All Tests

Open Test Explorer (Test menu → Test Explorer) and click Run All. All 13 tests should show green ticks.

[

```
Test Explorer - Full Run
========================

CalcLibrary.Tests.MathLibraryTests
Passed TestSubtract(10, 3, 7) 5 ms
Passed TestSubtract(20, 10, 10) 4 ms
Passed TestSubtract(0, 5, -5) 3 ms
Passed TestSubtract(-4, -2, -2) 3 ms
Passed TestMultiply(3, 4, 12) 4 ms
Passed TestMultiply(5, 5, 25) 3 ms
Passed TestMultiply(-2, 3, -6) 3 ms
Passed TestMultiply(0, 10, 0) 3 ms
Passed TestDivide(10, 2, 5) 5 ms
```

```
Passed TestDivide(9, 3, 3) 4 ms
Passed TestDivide(-6, 2, -3) 3 ms Passed
TestDivide_ByZero_ThrowsException 6 ms
Passed TestAddAndClear 7 ms
=======================================
Total: 13 Passed: 13 Failed: 0 Skipped: 0
Test Run Successful.
```